PyPDFLoader — Student Notes

1. Important PyPDFLoader Syntax Summary

Requirement	Syntax
Import loader	from langchain_community.document_loaders import PyPDFLoader
Create PDF loader	PyPDFLoader(file_path="sample.pdf")
Page-wise loading	mode="page"
Combined loading	mode="single"
Load all Documents	loader.load()
Iterate over Documents	loader.lazy_load()
Read page text	document.page_content
Read page index	document.metadata["page"]
Read page count	document.metadata["total_pages"]

Read page label	document.metadata.get("page_label")
Read source	document.metadata["source"]
Page separator	pages_delimiter="..."

2. Easy Memory Formula

PDF File

↓

PyPDFLoader

↓

load() / lazy_load()

↓

Document Objects

↓

page_content + metadata

3. Program 1 — Our First PyPDFLoader Program

Objective

Load a single-page PDF and access:

- Return type
- Total Documents
- Page content
- Metadata
- Source information
- Page index
- Total pages

```
from langchain_community.document_loaders import (
   PyPDFLoader
)



# ========================================
# CREATE PDF LOADER
# ========================================

loader = PyPDFLoader(
```

```
    file_path="library_notice.pdf"
)



# ========================================
# LOAD PDF DOCUMENTS
# ========================================

documents = loader.load()



# ========================================
# DISPLAY RETURN INFORMATION
# ========================================

print("RETURN TYPE")
print("-----------")

print(
    type(documents).__name__
```

```
)  # list


print("\nTOTAL DOCUMENTS")
print("---------------")

print(
    len(documents)
)  # 1


# ========================================
# ACCESS FIRST DOCUMENT
# ========================================

document = documents[0]


# ========================================
# DISPLAY EXTRACTED CONTENT
```

```
# ========================================

print("\nPAGE CONTENT")
print("------------")

print(
    document.page_content
)


# ========================================
# DISPLAY METADATA
# ========================================

print("\nMETADATA")
print("--------")

print(
    document.metadata
)
```

```
# ========================================
# DISPLAY SOURCE INFORMATION
# ========================================

print("\nSOURCE FILE")
print("-----------")

print(
    document.metadata["source"]
)



# ========================================
# DISPLAY PDF PAGE INFORMATION
# ========================================

print("\nPAGE INFORMATION")
print("----------------")
```

```
print(
    "Page Index:",
    document.metadata["page"]
)  # 0


print(
    "Total Pages:",
    document.metadata["total_pages"]
)
```

4. Understanding Program 1

The important line is:

documents = loader.load()

This returns:

List of Document Objects

For a single-page PDF:

documents

↓

[

Document(...)

]

So:

document = documents[0]

gives the first Document.

---

5. documents vs document

documents

↓

List

documents[0]

↓

Document Object

Very important:

documents.page_content

is wrong because documents is a list.

Correct:

documents[0].page_content

6. Accessing Page Content

document.page_content

gives the extracted text from the PDF page.

Simple:

Document

↓

page_content

↓

Extracted Text

7. Accessing Metadata

document.metadata

may contain information such as:

source

page

page_label

total_pages

Example:

```
{
   "source": "library_notice.pdf",
```

```
    "page": 0,
    "page_label": "1",
    "total_pages": 1
}
```

8. Understanding Page Index

document.metadata["page"]

uses zero-based indexing.

Actual Page	Page Index
Page 1	0
Page 2	1
Page 3	2

Therefore:

page = 0

means:

First PDF Page

9. Program 1 Flow

library_notice.pdf

↓

PyPDFLoader

↓

loader.load()

↓

documents

↓

documents[0]

↓

Document

|

├── page_content

└── metadata

10. Program 2 — Loading a Multipage PDF

Objective

Load a three-page PDF and access each page separately.

from langchain_community.document_loaders import (

```
    PyPDFLoader
)



# ========================================
# CREATE PAGE-WISE PDF LOADER
# ========================================

loader = PyPDFLoader(
    file_path="company_policy.pdf",
    mode="page"
)



# ========================================
# LOAD PDF PAGES
# ========================================

documents = loader.load()
```

```
# ========================================
# DISPLAY TOTAL DOCUMENTS
# ========================================

print("COMPANY POLICY PDF")
print("==================")


print(
    "Total Documents:",
    len(documents)
)


# ========================================
# DISPLAY EACH PAGE
# ========================================

for document in documents:
```

```
page_index = (
    document.metadata["page"]
)

page_number = (
    page_index + 1
)


print(
    "\nPAGE",
    page_number
)

print(
    "--------------------"
)

print(
```

```
        "CONTENT:\n",
        document.page_content
    )
```

11. Understanding mode="page"

mode="page"

means:

Each PDF Page

↓

Separate Document

Suppose:

company_policy.pdf

↓

3 Pages

Then:

documents

↓

Document 1 → Page 1

Document 2 → Page 2

Document 3 → Page 3

So:

len(documents)

will normally be:

3

12. Why page_index + 1?

We get:

page_index = document.metadata["page"]

For first page:

page_index = 0

But for display, we usually want:

PAGE 1

Therefore:

page_number = page_index + 1

Example:

Index 0 → Page 1

Index 1 → Page 2

Index 2 → Page 3

13. Program 2 Flow

company_policy.pdf

↓

mode="page"

↓

loader.load()

↓

documents

↓

Loop Through Documents

↓

Page 1

Page 2

Page 3

14. Program 3 — Understanding PDF Metadata in Detail

Objective

Examine:

- Source metadata
- PDF page index

- Display page number
- Page label
- Total page count

```
from langchain_community.document_loaders import (
    PyPDFLoader
)

from pathlib import Path



# ========================================
# CREATE PDF LOADER
# ========================================

loader = PyPDFLoader(
    file_path="company_policy.pdf",
    mode="page"
)
```

```
# ========================================
# LOAD DOCUMENTS
# ========================================

documents = loader.load()



# ========================================
# DISPLAY PDF SUMMARY
# ========================================

print("PDF METADATA REPORT")
print("====================")



print(
    "Total Documents:",
    len(documents)
)
```

```
# ========================================
# DISPLAY METADATA FOR EACH PAGE
# ========================================

for document in documents:

    metadata = document.metadata


    print("\nPAGE METADATA")
    print("-------------")


    print(
        "Source:",
        metadata["source"]
    )
```

```
print(
    "Page Index:",
    metadata["page"]
)


print(
    "Display Page:",
    metadata["page"] + 1
)


print(
    "Page Label:",
    metadata.get(
        "page_label",
        "Not available"
    )
)
```

```
print(
    "Total Pages:",
    metadata["total_pages"]
)
```

15. Why Store Metadata in a Variable?

Instead of repeatedly writing:

document.metadata["page"]

we can write:

metadata = document.metadata

Then:

metadata["page"]

This makes the code easier to read.

16. Important Metadata Fields

Source

metadata["source"]

gives the PDF source.

Example:

company_policy.pdf

Page Index

metadata["page"]

gives:

Zero-Based Page Index

Example:

0

1

2

Display Page

metadata["page"] + 1

gives:

1

2

3

Page Label

```
metadata.get(
    "page_label",
    "Not available"
)
```

This safely reads the page label.

If it is unavailable:

Not available

is returned.

Total Pages

```
metadata["total_pages"]
```

gives the total number of pages in the PDF.

Example:

3

17. Metadata Memory Formula

source

↓

Which PDF?

page

↓

Which Page Index?

page_label

↓

What Is the PDF Page Label?

total_pages

↓

How Many Pages?

18. Program 3 Flow

PDF

↓

PyPDFLoader

↓

Documents

↓

Loop Through Each Document

↓

metadata

|

├── source

├── page

├── page_label

└── total_pages

19. Program 4 — mode="page" vs mode="single"

```
from langchain_community.document_loaders import (
    PyPDFLoader
)



# ========================================
# PAGE MODE
# ========================================

page_loader = PyPDFLoader(
    file_path="company_policy.pdf",
    mode="page"
)



page_documents = (
    page_loader.load()
)
```

```
# ========================================
# SINGLE MODE
# ========================================

single_loader = PyPDFLoader(
    file_path="company_policy.pdf",
    mode="single",
    pages_delimiter=
    "\n--- PAGE BREAK ---\n"
)


single_documents = (
    single_loader.load()
)


# ========================================
# COMPARE DOCUMENT COUNTS
```

```
# ========================================

print(
   "PAGE MODE DOCUMENTS"
)

print(
   "------------------"
)

print(
   len(page_documents)
)



print(
   "\nSINGLE MODE DOCUMENTS"
)

print(
```

```
    "--------------------"
)

print(
    len(single_documents)
)


# ========================================
# ACCESS COMBINED DOCUMENT
# ========================================

document = (
    single_documents[0]
)


# ========================================
# CHECK PAGE METADATA
# ========================================
```

```
print(
    "\nSINGLE MODE METADATA"
)

print(
    "--------------------"
)



print(
    "Total Pages:",
    document.metadata["total_pages"]
)



print(
    "Page Index Available:",
    "page" in document.metadata
)
```

```
# ========================================
# DISPLAY COMBINED PAGE CONTENT
# ========================================

print(
    "\nCOMBINED PDF CONTENT"
)

print(
    "--------------------"
)



pages = (
    document.page_content.split(
        "--- PAGE BREAK ---"
    )
)
```

```
for number, text in enumerate(
    pages,
    start=1
):

    clean_text = " ".join(
        text.split()
    )


    print(
        "\nSECTION",
        number
    )

    print(
        clean_text
    )
```

20. Understanding Page Mode

mode="page"

Suppose the PDF contains three pages.

Then:

Page 1 → Document 1

Page 2 → Document 2

Page 3 → Document 3

Therefore:

3 Pages

↓

3 Documents

21. Understanding Single Mode

mode="single"

means:

Page 1

Page 2

Page 3

↓

Combined

↓

One Document

Therefore:

3 Pages

↓

1 Document

22. Page Mode vs Single Mode

Page Mode	**Single Mode**
mode="page"	mode="single"
Separate Document per page	One combined Document
Page-wise processing	Whole PDF processing
Multiple Documents	One Document
Page metadata available per page	Combined document metadata

Easy memory:

PAGE

↓

Separate

SINGLE

↓

Combine

23. What Is pages_delimiter?

We use:

pages_delimiter=

"\n--- PAGE BREAK ---\n"

This places a separator between page texts in single mode.

Example:

Page 1 Text

--- PAGE BREAK ---

Page 2 Text

--- PAGE BREAK ---

Page 3 Text

Important:

pages_delimiter

↓

Separates Page Text

Inside One Document

It does **not** create multiple Documents.

24. Understanding split()

We use:

```
pages = document.page_content.split(
    "--- PAGE BREAK ---"
)
```

In Python:

split()

returns:

list[str]

Example:

```
text = (
    "Page 1"
    "--- PAGE BREAK ---"
    "Page 2"
)
```

After:

```
text.split(
    "--- PAGE BREAK ---"
)
```

we get conceptually:

```
[
    "Page 1",
    "Page 2"
]
```

So:

split()

↓

String

↓

List of Strings

25. Understanding enumerate()

```
for number, text in enumerate(
   pages,
   start=1
):
```

enumerate() gives:

Number + Value

Example:

1 → Page 1 Text

2 → Page 2 Text

3 → Page 3 Text

Because:

start=1

counting begins from 1.

26. Cleaning Extracted Text

We use:

```
clean_text = " ".join(
   text.split()
)
```

First:

text.split()

splits based on whitespace.

Then:

" ".join(...)

joins the pieces using one space.

Example:

"Python

    is   easy"

becomes:

Python is easy

So:

split()

  ↓

Remove irregular whitespace

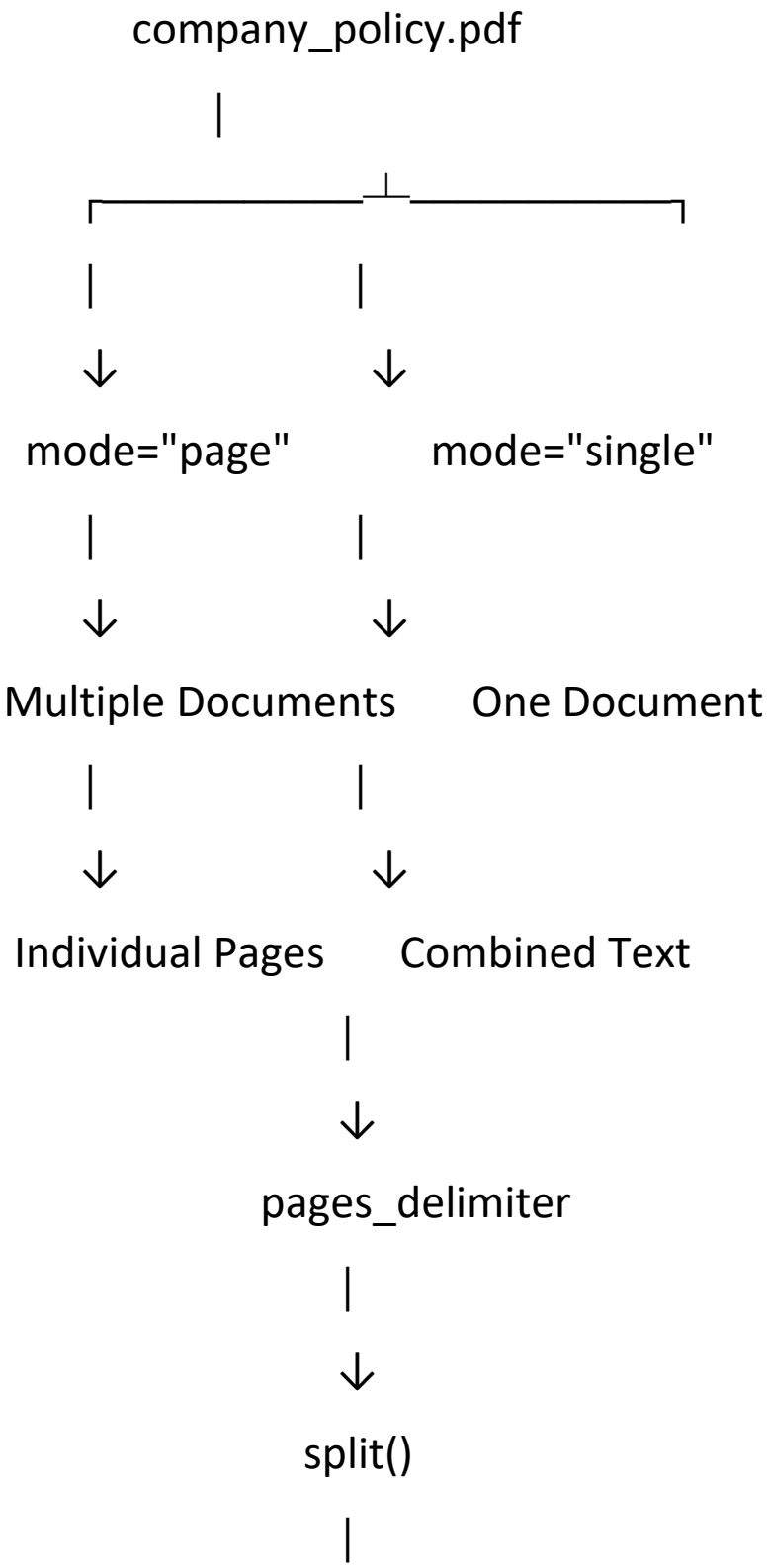
and:

join()

  ↓

Join with one space

27. Program 4 Main Architecture



↓

List of Sections

28. Program 5 — Page-Wise Lazy Loading and Empty Text Detection

Objective

Process PDF pages using:

lazy_load()

and identify pages containing no extractable text.

```
from langchain_community.document_loaders import (
    PyPDFLoader
)



# ========================================
# CREATE PAGE-WISE PDF LOADER
# ========================================

loader = PyPDFLoader(
    file_path="company_policy.pdf",
```

```
    mode="page"
)


# ========================================
# PROCESS PDF PAGES LAZILY
# ========================================

processed_pages = 0


for document in loader.lazy_load():

    processed_pages += 1


    page_number = (
        document.metadata["page"] + 1
    )
```

```
text = (
    document.page_content
)


print(
    "\nPAGE",
    page_number
)

print(
    "--------------------"
)


if text:

    print(
        "STATUS: Text extracted"
```

```
    )

    print(
        "CONTENT:",
        text
    )


else:

    print(
        "STATUS: No extractable text"
    )

    print(
        "Check whether the page is blank,"
        " scanned, or requires a different"
        " extraction approach."
    )
```

```
# ========================================
# DISPLAY TOTAL
# ========================================

print(
    "\nTOTAL PAGES PROCESSED"
)

print(
    "--------------------"
)

print(
    processed_pages
)
```

29. What Is lazy_load()?

loader.lazy_load()

allows us to process Documents as they are yielded.

Concept:

PDF

↓

lazy_load()

↓

Document 1

↓

Process

↓

Document 2

↓

Process

↓

Document 3

↓

Process

30. load() vs lazy_load()

load()

documents = loader.load()

Concept:

Load Everything

↓

Return Complete List

lazy_load()

for document in loader.lazy_load():

Concept:

Yield Document

↓

Process

↓

Yield Next

31. Simple Comparison

load()	**lazy_load()**
Returns collection/list	Yields Documents
Load first, process later	Process as yielded
Easy for normal loading	Useful for iterative loading

Easy memory:

load()

↓

COLLECT

lazy_load()

↓

PROCESS ONE BY ONE

32. Counting Processed Pages

Initially:

processed_pages = 0

Every time one Document is received:

processed_pages += 1

Suppose three pages are processed:

Start = 0

Page 1

↓

1

Page 2

↓

2

Page 3

↓

3

33. Detecting Extracted Text

text = document.page_content

Then:

if text:

means:

Text Available?

If yes:

STATUS: Text extracted

Otherwise:

STATUS: No extractable text

34. Why Might a Page Have No Extractable Text?

Possible reasons include:

Blank Page

Scanned Page

Different Extraction Approach Required

So:

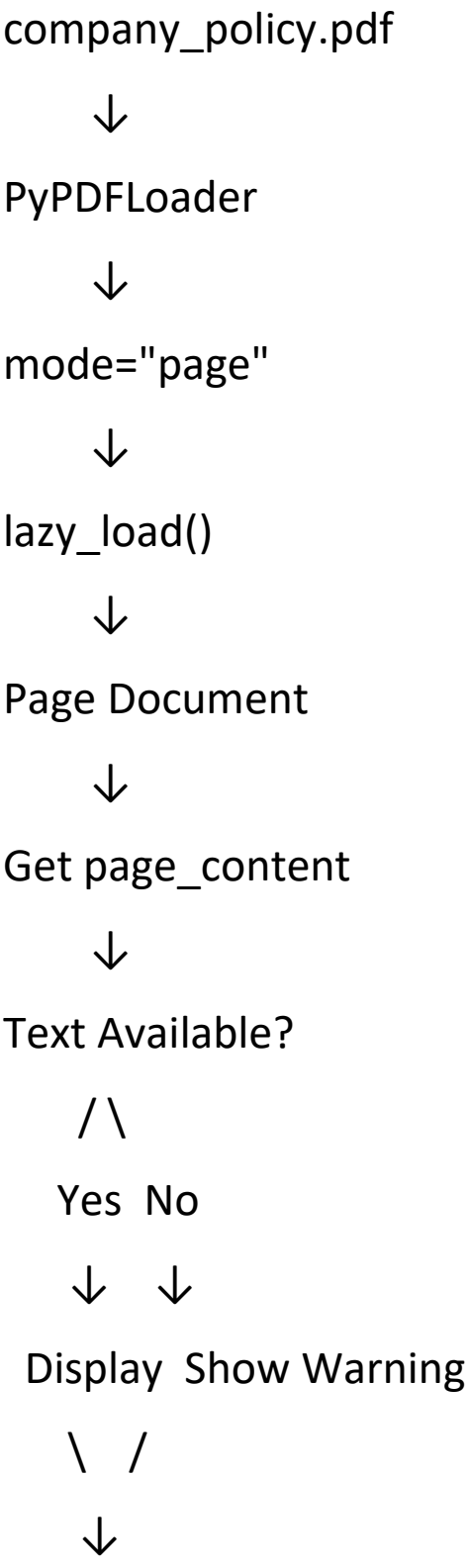
No Extracted Text

≠

No Visible Information

The page may still contain an image or scanned text.

35. Program 5 Flow



Next Page

36. Five Programs — Quick Revision

Program 1

Basic PDF Loading

↓

Content + Metadata

Program 2

Multipage PDF

↓

One Document per Page

Program 3

Metadata

↓

source

page

page_label

total_pages

Program 4

page mode

VS

single mode

Program 5

lazy_load()

↓

Page-Wise Processing

↓

Detect Empty Text

37. Most Important Code Lines

Create PDF Loader

```
loader = PyPDFLoader(
   file_path="company_policy.pdf"
)
```

Page Mode

```
mode="page"
```

Single Mode

```
mode="single"
```

Load

```
documents = loader.load()
```

Lazy Load

loader.lazy_load()

Content

document.page_content

Page Index

document.metadata["page"]

Total Pages

document.metadata["total_pages"]

Page Label

```
document.metadata.get(
   "page_label"
)
```

38. Common Mistake 1 — Thinking .load() Returns One Document

Wrong:

loader.load()

↓

Document

Correct:

loader.load()

↓

List of Documents

So:

documents = loader.load()

and:

document = documents[0]

39. Common Mistake 2 — Thinking Page Index Starts from 1

Wrong:

First Page

↓

page = 1

Correct:

First Page

↓

page = 0

For display:

page_number = (

document.metadata["page"]

+ 1

)

40. Common Mistake 3 — Thinking mode="single" Means One Page

No.

It means:

All PDF Pages

↓

Combined

↓

One Document

41. Common Mistake 4 — Thinking pages_delimiter Creates Documents

No.

pages_delimiter="--- PAGE BREAK ---"

only separates page text inside the combined Document.

42. Common Mistake 5 — Thinking split() Returns a String

No.

text.split(...)

returns:

List of Strings

Memory:

split

↓

One String

to

Many Strings

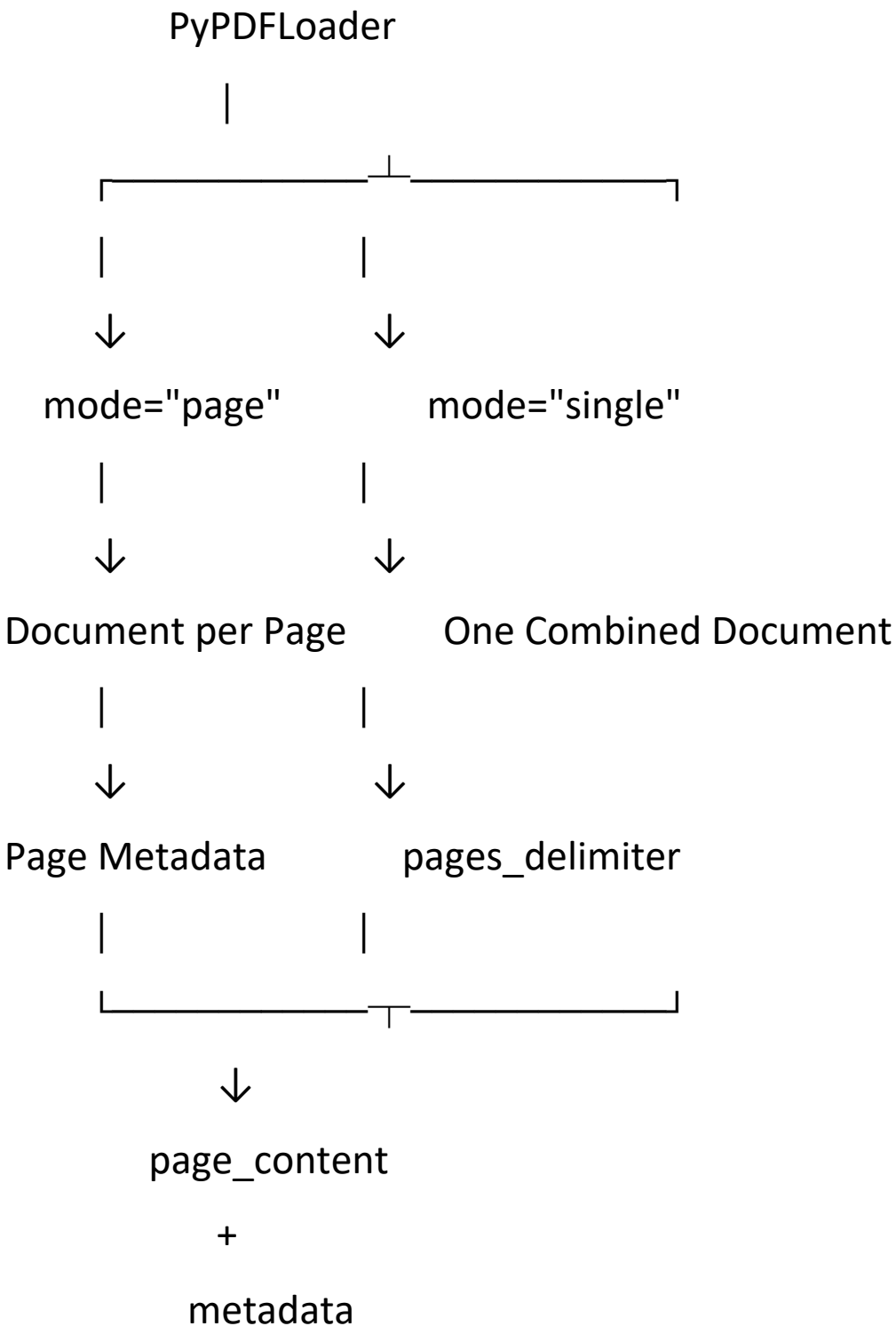
43. Common Mistake 6 — Thinking lazy_load() Returns the Complete List

lazy_load() is used in a loop:

for document in loader.lazy_load():

It allows Documents to be processed as they are yielded.

44. Final Comparison



45. Final Memory Formula

PDF

↓

PyPDFLoader

↓

Document Objects

↓

page_content + metadata

Page mode:

PAGE

↓

SEPARATE

Single mode:

SINGLE

↓

COMBINE

Lazy loading:

lazy_load()

↓

PROCESS ONE BY ONE

46. One-Line Conclusion

PyPDFLoader converts PDF content into LangChain Document objects; page mode creates page-wise Documents, single mode combines pages into one Document, and lazy_load() allows Documents to be processed one by one.